Open in Colab

Iterables

In [22]:
```
w = 'hello', 'world', '!'
h = 'hello'
```

Operations on Iterables

You can get the length of a tuple by the builtin `len()` function.

In [23]:
```
len(w), len(h)
```

Out[23]:
```
(3, 5)
```

To test if an item is in the iterable use `in` keyword.

In [24]:
```
'world' in w, 'hello' in w
```

Out[24]:
```
(True, True)
```

To test if an item is **not** in the iterable use `not in` keyword.

In [25]:
```
'world' not in w, 'hello' not in w
```

Out[25]:
```
(False, False)
```

To count the number of occurences of an item in a iterable use `.count()` method.

In [26]:
```
w.count('world'), w.count('hello')
```

Out[26]:
```
(1, 1)
```

Unpacking iterables

If you need you can assign different variables to each item of an iterable. This is called unpacking.

In [27]:
```
w
```

Out[27]:
```
('hello', 'world', '!')
```

In [28]:
```
first, second, third = w
```

In [29]:
```
first
```

Out[29]:
```
'hello'
```

In [30]:
```
second
```

Out[30]:
```
'world'
```

In [31]:
```
third
```

```
Out[31]: '!'
```

Also you can do this nested if the iterable is nested.

```
In [38]: (f1, f2, f3, f4, f5), second, third = w
```

```
In [39]: f1
```

```
Out[39]: 'h'
```

Repacking Tuple Items

You can use `*` to unpack a tuple and use `,` to pack to a tuple. Here `*` prevents Python from creating a nested tuple instead of a flat one.

```
In [41]: *second, None
```

```
Out[41]: ('w', 'o', 'r', 'l', 'd', None)
```

Compare this with:

```
In [42]: second, None
```

```
Out[42]: ('world', None)
```